Statistique inférentielle

Vincent Le Meur

2023

TP3 :Tests d'hypothèse sous R

Compte rendu

Question 1

Générer et stocker dans une matrice 1000 échantillons de taille n = 30 à partir de la population de données initiale

Creation de la matrice:

```
mat<- matrix(sample(sinistres$montant, size=30, replace=TRUE),nrow = 1000, ncol =30)
```

Question 2

Écrire une fonction permettant d'effectuer un test de nullité de la moyenne. Cette fonction aura en paramètre un échantillon de données, l'écart-type σ et le risque de première espèce α, et retournera 1 si H0 est rejetée et 0 sinon.

On rappelle que si la valeur de la statistique est supérieur à la valeur critique on peut rejeter l'hypothèse H_0 de nullité de la moyenne:

La formule de la statistique :

$$T = \frac{\bar{X}_n - m_0}{\sigma/\sqrt{n}}$$

```
test_moyenne <- function(echantillon, sigma, alpha) {
  n <- length(echantillon)
  moyenne <- mean(echantillon)

  valeur_critique <- qnorm(1 - alpha/2)  # valeur critique


  statistique <- (moyenne-0) / (sigma / sqrt(n)) #statistique de test

  # Test de nullité de la moyenne
  if (abs(statistique) > valeur_critique) {
    return(1)
  } else {
    return(0)
  }
}
```

Question 3

Parmi les 1000 échantillons générés, combien de fois peut-on affirmer au risque $\alpha = 5\%$ que l'indemnisation moyenne est différente de 650 euros.

On change la fonction d'avant en changeant 0 par 650:

```
test_moyenne2 <- function(echantillon, sigma, alpha) {
  n <- length(echantillon)
  moyenne <- mean(echantillon)

  valeur_critique <- qnorm(1 - alpha/2)  # valeur critique


  statistique <- (moyenne-650) / (sigma / sqrt(n)) #statistique de test

  # Test de nullité de la moyenne
  if (abs(statistique) > valeur_critique) {
    return(1)
  } else {
    return(0)
  }
}
```

On réutilise la fonction précédente pour trouver le nombre

```
sigma = 60
alpha = 0.05

res = apply(mat,1,test_moyenne2,sigma,alpha)
nbr_rejet <-length(which(res==1))
```

Table 1: Nombre de rejets

n	Nombre_rejets
30	400

Question 4

Retrouver la formule explicite de la p-value de ce test. Écrire une fonction qui à partir d'un échantillon de données et de l'écart-type σ calcule la p-value du test.

Dans le cas où l'écart-type de la population est connu (σ), et en supposant une distribution normale, la statistique de test suit une distribution normale centrée réduite

La formule explicite de la p-value de ce test est:

$$p - value = 2 * (1 - \phi(|statistique|)$$

Avec comme statistique:

$$\frac{moyenne\ echantillon - m_0}{\sigma/\sqrt{n}}$$

```
p_value <- function(X,moyenne, sigma){
  statistique <- (mean(X)-moyenne)/(sigma/sqrt(length(X)))
```

```
  p_val <- 2*(1-pnorm(abs(statistique)))
  return(p_val)
}
sigma = 60
p_values <- apply(mat, 1, p_value,650, sigma)
```

Question 5

Retrouver la formule explicite de la puissance de ce test. Écrire une fonction qui à partir de n, de σ, de σ et de $\delta = \mu_1$ - μ_0 (où μ_1 est la "vraie" valeur de μ sous H1) calcule la puissance du test.

La formule de la puissance du test est:

$$P = 1 - \beta = 1 - P\left(q_{\frac{\alpha}{2}} - \delta * \frac{\sqrt{n}}{\sigma/\sqrt{n}}\right) + P\left(-q_{\frac{\alpha}{2}} - \delta * \frac{\sqrt{n}}{\sigma/\sqrt{n}}\right)$$

```
puissance <- function(n,sigma,alpha,delta){
  return(1 - pnorm((qnorm(1 - alpha/2) - delta * sqrt(n)) / (sigma / sqrt(n))) + pnorm((-qnorm(1 - alpha

}

puis <- puissance(30,sigma,alpha,650)
```

Table 2: Puissance du test

n	Puissance
30	1

Question 6

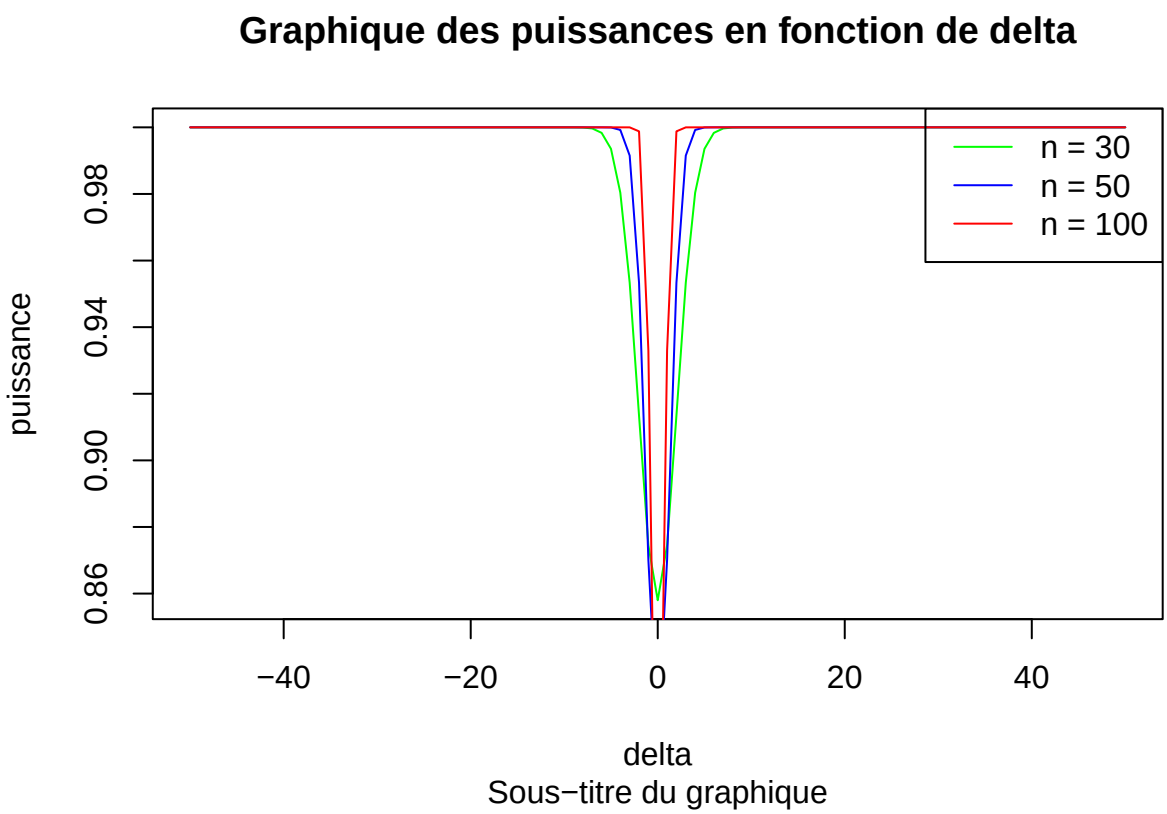
En fixant $\alpha = 5\%$, tracer la puissance du test pour n = 30, 50 et 100 en fonction de $\delta \in$[-50, 50].

Schéma

```
alpha=0.05
alpha <- 0.05
n <- c(30, 50, 100)
delta <- seq(-50, 50, by = 1)
l <- length(delta)

# Initialiser les vecteurs d'ordonnées pour chaque valeur de n
ordonnees_30 <- delta
ordonnees_50 <- delta
ordonnees_100 <- delta

# Calculer les ordonnées pour chaque valeur de n
for (i in 1:l) {
  ordonnees_30[i] <- puissance(n[1], sigma, alpha, delta[i])
  ordonnees_50[i] <- puissance(n[2], sigma, alpha, delta[i])
  ordonnees_100[i] <- puissance(n[3], sigma, alpha, delta[i])
}
```



Question 7

Refaire les questions 5 et 6 en utilisant la fonction pwr.norm.test disponible dans le package

pwr. On rappelle que la fonction pour charger un package déjà installé sur le système est library, et que des nouveaux packages peuvent être téléchargés et installés à partir des dépôts CRAN à l'aide de la fonction install.packages

Q5

```
library(pwr)

n <- 30
sigma <- 60
alpha <- 0.05
delta <- 650  # (mu1 - mu0)


Puissance <- pwr.norm.test(n =n, d = delta / sigma, sig.level = alpha, alternative = "two.sided")$power
```

Table 3: Puissance avec pwr.norm.test

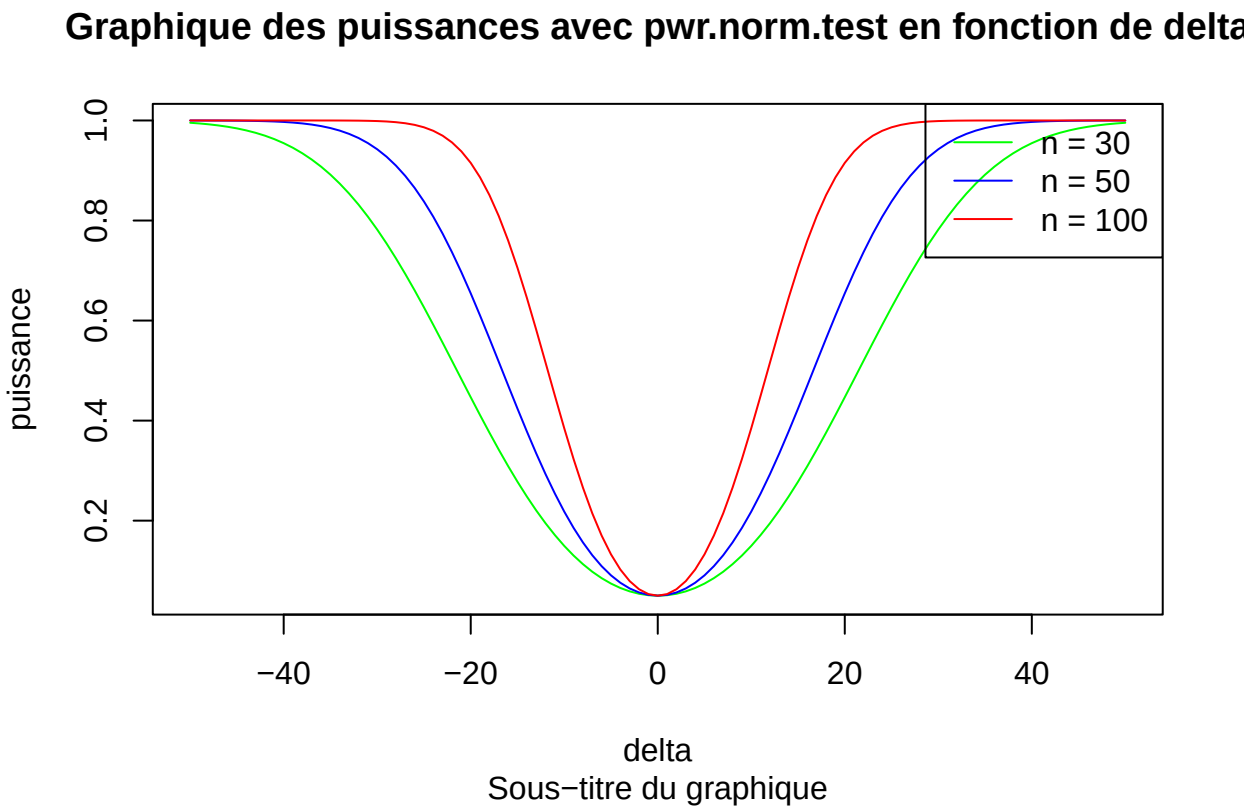
n	Puissance
30	1

Q6

```
alpha=0.05
alpha <- 0.05
n <- c(30, 50, 100)
delta <- seq(-50, 50, by = 1)
l <- length(delta)

# Initialiser les vecteurs d'ordonnées pour chaque valeur de n
ordonnees_30 <- delta
ordonnees_50 <- delta
ordonnees_100 <- delta

# Calculer les ordonnées pour chaque valeur de n
for (i in 1:l) {
  ordonnees_30[i] <- pwr.norm.test(n =n[1], d = delta[i] / sigma, sig.level = alpha, alternative = "two
  ordonnees_50[i] <- pwr.norm.test(n =n[2], d = delta[i] / sigma, sig.level = alpha, alternative = "two
  ordonnees_100[i] <- pwr.norm.test(n =n[3], d = delta[i] / sigma, sig.level = alpha, alternative = "two
}
```



Question 8

En fixant $\alpha = 5\%$, calculer le nombre d'observations nécessaires pour que le risque de seconde espèce ne dépasse pas 10% pour $\delta = 20$.

```
library(pwr)

alpha <- 0.05
delta <- 20
```

```r
puiss <- 0.9

# Calcul du nombre d'observations nécessaires
n <- ceiling(pwr.norm.test(d = delta/sigma, power = 0.9, sig.level = alpha, alternative = "two.sided")$n
```

Table 4: nombre d'observations nécessaires pour que le risque de seconde espèce ne dépasse pas 10%

Nombre_d_observation
95